FlutterShy User Manual and Theory

Alexander Ketzle

October 2026

Contents

1 Introduction **3**

2 User Manual **4**
2.1 User Inputs . . . 4
2.1.1 Fin Parameters . . . 4
2.1.2 Simulation Parameters . . . 5
2.1.3 Advanced Simulation Controls . . . 5
2.2 Outputs . . . 7
2.2.1 Text Report . . . 7
2.2.2 Plots . . . 7

3 Operating Theory **8**
3.1 Calculation of Fin Parameters . . . 8
3.1.1 Fin Average Chord, c and Fin Average Semichord, b . . . 10
3.1.2 Fin Volume, V . . . 10
3.1.3 Fin Polar Moment of Inertia, J_0 . . . 11
3.1.4 Fin Reduced Radius of Gyration, $\overline{r}$ . . . 12
3.1.5 Fin Mass Per Unit Span, m . . . 12
3.1.6 Fin Bending Natural Frequency, ω_h . . . 12
3.1.7 Fin Torsional Natural Frequency, ω_α . . . 12
3.2 Subsonic Flutter Prediction Using the $1/k$ Method . . . 13
3.2.1 Subsonic Compressibility Correction . . . 14
3.3 Supersonic Flutter Prediction Using the Kearns Supersonic Method . . . 15

1 Introduction

This document comprises the user manual and theory explanation for *FlutterShy*. This documents expects the reader to be familiar with basic MATLAB programming in order to operate *FlutterShy* and expects the reader to become familiar with the underlying theory used for the calculations performed by *FlutterShy*.

2 User Manual

FlutterShy is intended to be used for high-power amateur rocketry by amateur rocketeers as a fin flutter prediction software. It is written in MATLAB and works on versions as old as R2023a. It requires user inputs on the physical properties of the fins, a RASAero II flight simulation file, and launch site altitude.

2.1 User Inputs

FlutterShy has a total of 15 parameters intended for users to manipulate. Nine parameters correspond to physical properties of the fin, two parameters correspond to the RASAero II simulation, and four are optional parameters that modify advanced prediction behavior. *FlutterShy* reads an input text file for the parameters, and an example is included in the repository as `InputExample.txt`.

2.1.1 Fin Parameters

FlutterShy includes the function `calculateFinProperties` which takes in a user-defined fin and calculates most of the properties needed from the physical properties of the fin. For more information on the function, see section 3.1.

The first fin parameter is c, the average chord of the fin, in feet. This length is measured from the leading (forward) edge of the fin to the trailing (rearward) edge of the fin, parallel to the body of the rocket.

The second fin parameter is m, the mass per unit span of the fin, in slugs/foot. This parameter is calculated by taking the mass of the fin in slugs and dividing it by the span of the fin, in feet. Note the usage of the slugs unit; This is due to the atmospheric model script used in *FlutterShy*. One slug is a mass equivalent to 32.17405 lb in standard gravity.

The third fin parameter is $\overline{x}$ (in the code as x_bar), the chord-normalized distance of the center of gravity (c.g.) of the fin to the leading edge, a ratio value. This parameter is found by finding the location of the c.g. on the fin, then taking the chord-wise distance from the leading edge of the fin to the c.g. and dividing it by the chord at the span location of the c.g. on the fin. A value of 0 corresponds to a c.g. at the leading edge of the fin, a value of 0.5 corresponds to a c.g. centered between the leading and trailing edges, and a value of 1 corresponds to a c.g. located at the trailing edge. Most fins will have a value close to or exactly 0.5.

The fourth fin parameter is $\overline{r}$ (in the code as r_bar), the semichord-normalized radius of gyration of the fin, a ratio value. This value is found by taking the radius of gyration of the fin about the elastic axis and dividing it by the average semichord (half of the average chord).

The fifth fin parameter is ω_h (in the code as freq_h), the bending frequency of the fin in radians/second. This value can be found several ways. The author recommends either vibration testing the fin of interest under appropriate conditions, or performing modal analysis using the finite element analysis (FEA) suite of choice for the reader, assuming appropriate boundary conditions. A method of direct calculation can be found on page 40 of [1].

The sixth fin parameter is ω_α (in the code as freq_alpha), the torsion frequency of the fin in radians/second. Just like ω_h, this value can be found several ways. The author recommends either vibration testing the fin of interest under appropriate conditions, or performing modal analysis using the finite element analysis (FEA) suite of choice for the reader, assuming appropriate boundary conditions. A method of direct calculation can be found on page 40 of [1].

The seventh fin parameter is a_h, the chord-normalized distance of the elastic axis (e.a.) of the fin from the leading edge, a ratio value. This parameter is found similarly to $\overline{x}$, instead finding the location of the e.a., and taking the average value along the fin. A value of 0 corresponds to an e.a. at the leading edge of the fin, a value of 0.5 corresponds to an e.a. centered between the leading and trailing edges, and a value of 1 corresponds to an e.a. located at the trailing edge. Most fins will have a value close to or exactly 0.5.

The eighth and ninth fin parameters are g_h and g_α (in the code as g_h and g_alpha respectively), the bending and torsional structural damping coefficients, respectively. These values are generally determined experimentally, however data does not exist for amateur rocketry applications. Values of 0.03 to 0.05 are generally used in the aviation industry, however values in the range of 0.003 to 0.005 may be more appropriate for amateur rocketry.

2.1.2 Simulation Parameters

The first simulation parameter is the launch site sea level altitude in feet, which is in the code as site_altitude. This value is used to calibrate the atmospheric model used in calculations to provide accurate values derived from Mach number.

The second simulation parameter is the file path string to the RASAero II simulation data file in .csv format. This is in the code as RAS_Filepath. This is needed to provide the flight conditions needed for flutter calculation.

2.1.3 Advanced Simulation Controls

The controls presented here are intended to only be modified by advanced users who fully understand what these controls do and how they affect generated solutions. Most users should not have to change these controls from their default values. Changing these controls can significantly affect the performance and accuracy of *FlutterShy*.

The first advanced control is the size of the $1/k$ steps, in the code as `invkstepsize`. By default this value is 0.0001. This controls the fineness of the matrix generated for the $1/k$ method used for subsonic flutter solutions. Increasing the resolution (decreasing the magnitude of the value) from the default value causes a significant increase in *FlutterShy*'s runtime, while offering marginal accuracy increases. Reducing the resolution (increasing the magnitude of the value) will reduce *FlutterShy*'s runtime, at the cost of decreased accuracy, which could be significant in some cases.

The second advanced control is the maximum value of $1/k$ which *FlutterShy* tests to, in the code as `invkMax`. By default this value is 8. This controls the size of the matrix generated for the $1/k$ method used for subsonic flutter solutions. Increasing this value increases the size of the matrix generated by expanding the range of values for which the 1/k method is calculated. If *FlutterShy* is producing sensible results for subsonic flutter, modifying this value provides no benefit. The value should only be increased if flutter velocities seem extremely low for subsonic flight, as too small of a range can result in no intersection of the imaginary and real portions of the equations being generated, causing a value to be picked at an extrema that does not correspond physically. Generally, this value should not be increased past 14 unless necessary.

The third advanced control is the method handoff Mach number, in the code as `machGate`. By default this value is 1.01. This value controls the maximum flight Mach number at which *FlutterShy* will use subsonic methods instead of supersonic methods. By changing this value, the window of "subsonic" and "supersonic" flight is moved. The default value of 1.01 was selected due to the compressibility correction present in the Kearns supersonic method used by *FlutterShy*, which exhibits a singularity at Mach 1, or asymptotically approaches 0. It was found that the calculated values were generally stable starting near Mach 1.01, and thus at higher values corresponded to physical reality. Changing this value below 1 causes mathematical errors in calculation and produces imaginary flutter velocity numbers.

The fourth advanced control is the subsonic correction method selection, in the code as `subsonicCorrection`. By default, this value is "none". This controls if and which correction equation is applied to flutter velocity calculated within the "subsonic" regime as defined by `machGate`. The four options are:

- "tr685"
- "mat1"

- "mat2"
- "kearns"

For further information on the different corrections, see section 3.2.1.

2.2 Outputs

When *FlutterShy* is run, it will generate a text output and several plots to inform the user.

2.2.1 Text Report

The text report generated by *FlutterShy* is intended to provide a "quick-view" or summary of the calculated results, highlighting information that users are likely to be most interested in. The report is broken into three sections:

1. Minimum Flutter Factor of Safety
2. Minimum Flutter Velocity
3. Maximum Flight Velocity

The first section provides the minimum factor of safety, the flutter velocity at the minimum factor of safety, the flight velocity at the minimum factor of safety, and the time in flight when the minimum factor of safety occurs. The second section provides similar information as the first, adjusted for the minimum flutter velocity. The third section provides the maximum flight velocity, the flutter velocity at that point, and the flutter factor of safety at that point.

2.2.2 Plots

FlutterShy generates eight plots for the user when calculation is complete. These plots are:

1. Flutter Factor of Safety vs. Time
2. Flutter Velocity vs. Sim Velocity
3. Flutter Mach vs. Sim Mach
4. Mach Numbers vs. Time
5. Flutter Factor of Safety vs. Dynamic Pressure
6. Flutter Factor of Safety vs. Altitude
7. Flutter Factor of Safety vs. Sim Mach
8. Flutter Velocity vs. Time

These plots are intended to give a more complete view of the flutter behavior of the vehicle in relation to several parameters. These plots present all of the calculated flutter values over the flight to apogee. Flutter velocity is not calculated after apogee because it is assumed that the vehicle deploys a recovery system at such point, and flutter will cease to be relevant. This saves computational time and resources.

3 Operating Theory

FlutterShy calculates flutter velocity based off of the $1/k$ method found in [1, 2] for subsonic flight and the Kearns supersonic method found in [3], which is a simplification of [4] using piston theory. *FlutterShy* does not perform any form of FEA, CFD, or FSI, nor does it use the SDPM or DLM methods. This allows it to provide flutter information quickly and with acceptable accuracy for amateur rocketry, however it will have reduced accuracy compared to the more complicated methods mentioned previously.

At a high level, *FlutterShy* ingests the user input fin parameters, launch site information, and RASAero II simulation data, generates a series of vectors for the flight conditions at each time step, calculates the flutter velocity using the appropriate method, and then bundles the data to provide the report and plots. In its current iteration, *FlutterShy* has only been tested to work with a mass ratio vector input to the two calculation methods, requiring scalars in other parameters. Further, the $1/k$ method function does not natively handle vectors of the mass ratio correctly, and thus requires *FlutterShy* to pass it mass ratio values as scalars in a for loop, significantly increasing computation time. Future updates hope to address these shortcomings to make *FlutterShy* both faster and more useful, potentially allowing for parameter spaces to be tested using the software.

Both the subsonic and supersonic method use the same set of fin input parameters for calculation, differing only on their use of either flight Mach number or reduced frequency. The common symbols are defined as:

1. b, Fin average semichord
2. μ, the mass ratio
3. $\overline{x}$, the chord-normalized distance of the center of gravity from the leading edge
4. a_h, the chord-normalized distance of the elastic axis from the leading edge
5. $\overline{r}$, the semichord-normalized radius of gyration
6. ω_h, ω_α, the respective bending and torsion natural frequencies of the fin
7. g_h, g_α, the respective bending and torsion structural damping coefficients of the fin

furthermore, μ is defined in *FlutterShy* by

$$\mu = \frac{m}{\pi \rho b^2} \tag{1}$$

where m is the mass per unit span of the fin, and ρ is the air density of interest.

3.1 Calculation of Fin Parameters

FlutterShy provides the function `calculateFinProperties` which takes fin physical dimensions and outputs some of the necessary parameters for flutter calculation. This functions ingests a `fin` structure with required parameters `thickness, rootchord, tipchord, span, airfoil, density` and, if `airfoil = 'hexagonal'`, `chamferHeight`. The function then translates the parameters according to the following list:

- `thickness` $= t$
- `rootchord` $= c_r$
- `tipchord` $= c_t$
- `span`[1] $= h$
- `chamferHeight` $= h_c$

[1] Also referred to as the fin height

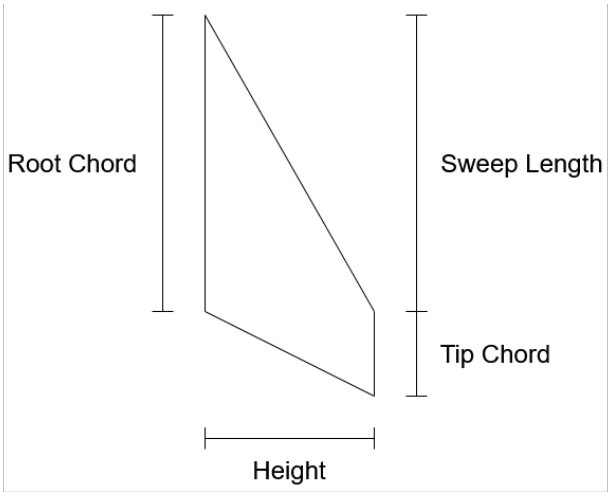
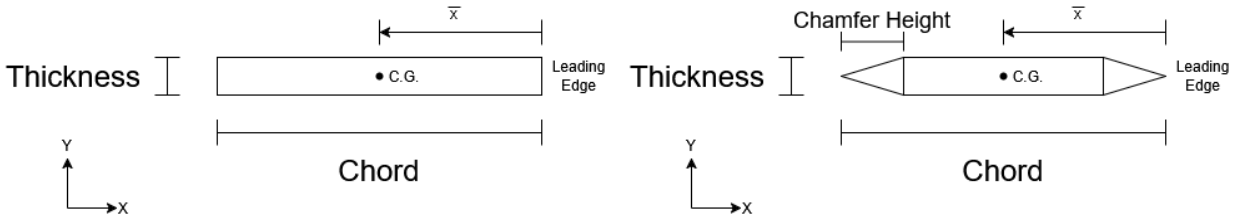
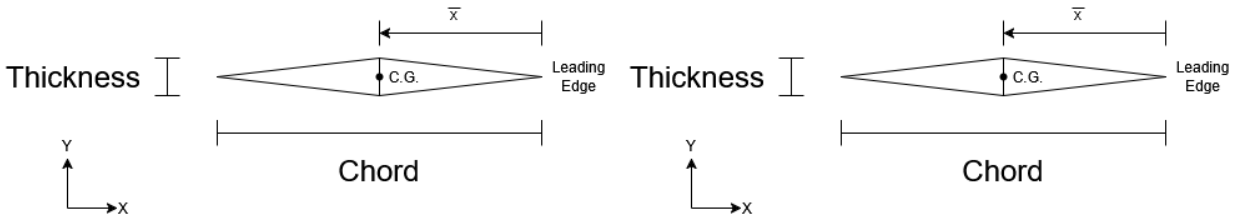


Figure 1: Fin Planform Dimensions



(a) Rectangular Airfoil (b) Hexagonal Airfoil

Figure 2: Airfoil dimensions for rectangular and hexagonal fins



(a) Diamond Airfoil (b) Biconvex Airfoil

Figure 3: Airfoil dimensions for diamond and biconvex fins

This function assumes that the fin meets certain assumptions:

1. The fin can be described as a trapezoid with a taper ratio $\lambda = \frac{c_t}{c_r}$ less than or equal to 1[2]
2. The thickness of the fin is constant
3. The location of greatest thickness is along the midchord line
4. The elastic axis and center of gravity of the fin lie along the midchord line
5. The chamfer length of hexagonal airfoils is constant along the span and equal for both the leading and trailing edge

If a fin does not follow all of the above assumptions, the calculations will no longer closely represent the real fin, and it is recommended to perform the calculations by hand.

3.1.1 Fin Average Chord, c and Fin Average Semichord, b

The average chord of the fin, c, is calculated by

$$c = \frac{c_r + c_t}{2} \tag{2}$$

The average semichord b is calculated by

$$b = \frac{c}{2} \tag{3}$$

and describes the chord length from the centerline of the fin to the trailing edge of the fin.

3.1.2 Fin Volume, V

The volume of the fin, V, is derived from

$$V = \int_0^h A_{airfoil} dx \tag{4}$$

For rectangular airfoils, this is

$$V = At \tag{5}$$

where

$$A = ch \tag{6}$$

For hexagonal airfoils, this is

$$V = (2A_{chamf}h) + \frac{((c_r - (2h_c)) + (c_t - (2h_c)))ht}{2} \tag{7}$$

where h_c is the chamfer length (assumed symmetric and constant) and

$$A_{chamf} = \frac{th_c}{2} \tag{8}$$

For diamond airfoils, this is

$$V = \frac{th(c_r + c_t)}{4} \tag{9}$$

For biconvex airfoils, this is

$$\frac{th(c_r + c_t)}{3} \tag{10}$$

[2]note that this includes triangular and rectangular planforms

3.1.3 Fin Polar Moment of Inertia, J_0

FlutterShy derives the Polar Moment of Inertia, J_0, from

$$J_0 = I_x + I_y \tag{11}$$

where I_x and I_y are the area moments of inertia with respect to the x and y axes, respectively.

For rectangular airfoils, this is

$$\frac{ct(c^2 + t^2)}{12} \tag{12}$$

For hexagonal airfoils,

$$I_x = I_{x_{rect}} + 2(I_{x_{chamf}} + A_{chamf}d) \tag{13}$$

$$I_y = I_{y_{rect}} + 2I_{y_{chamf}} \tag{14}$$

where

$$d = \overline{x}_{chamf} + \frac{b_{rect}}{2} \tag{15}$$

$$I_{x_{rect}} = \frac{b_{rect}t^3}{12} \tag{16}$$

$$I_{y_{rect}} = \frac{tb_{rect}^3}{12} \tag{17}$$

$$b_{rect} = c - 2h_c \tag{18}$$

$$I_{x_{chamf}} = \frac{h_c t^3}{48} \tag{19}$$

$$I_{y_{chamf}} = \frac{th_c^3}{36} \tag{20}$$

$$\overline{x}_{chamf} = \frac{h_c}{3} \tag{21}$$

$$A_{chamf} = \frac{th_c}{2} \tag{22}$$

For Diamond airfoils,

$$I_x = 2(I_{x_{chamf}} + A_{chamf}d) \tag{23}$$

$$I_y = 2I_{y_{chamf}} \tag{24}$$

where

$$d = \frac{h_c}{3} \tag{25}$$

$$I_{x_{chamf}} = \frac{h_c t^3}{48} \tag{26}$$

$$I_{y_{chamf}} = \frac{th_c^3}{36} \tag{27}$$

$$A_{chamf} = \frac{th_c}{2} \tag{28}$$

$$h_c = \frac{b}{2} \tag{29}$$

For Biconvex airfoils,

$$J_0 = I_x + I_y \tag{30}$$

where

$$I_x = \frac{4t^3 c}{105} \tag{31}$$

and

$$I_y = \frac{tc^3}{30} \tag{32}$$

3.1.4 Fin Reduced Radius of Gyration, $\overline{r}$

FlutterShy derives the Reduced Radius of Gyration, $\overline{r}$, from

$$\overline{r} = \sqrt{\frac{hJ_0}{b^2V}} \tag{33}$$

for rectangular airfoils, the formula simplifies to

$$\overline{r} = \sqrt{\frac{1 + (\frac{t}{c})^2}{3}} \tag{34}$$

3.1.5 Fin Mass Per Unit Span, m

The mass per unit span of the fin, m, is calculated by

$$m = \frac{V * \rho_{fin}}{h} \tag{35}$$

where ρ_{fin} is the density of the fin material.

3.1.6 Fin Bending Natural Frequency, ω_h

The natural bending frequency of the fin, ω_h, in radians per second is calculated by

$$\omega_h = \beta_1^2 \sqrt{\frac{EI_x}{mh^4}} \tag{36}$$

where m is the mass per unit span calculated above, $\beta_i = 1.87510407$, from page 108 of [5] as the first wave number for a clamped-free cantilever beam and E is the modulus of elasticity of the fin. For rectangular airfoils, I_x is calculated by

$$I_x = \frac{ct^3}{12} \tag{37}$$

For other airfoils, I_x is calculated as stated in section 3.1.3 above.

3.1.7 Fin Torsional Natural Frequency, ω_α

The natural torsional frequency of the fin, ω_α, in radians per second is calculated by (from page 193 of [5])

$$\omega_\alpha = \frac{\pi}{2} \sqrt{\frac{K_{torsion} G}{h^2 \rho J}} \tag{38}$$

where G is the shear modulus, and $K_{torsion}$ is calculated by (from [6])

$$K_{torsion} = \frac{c}{2} (\frac{t}{2})^3 (\frac{16}{3} - 3.36 \frac{t}{c} (1 - \frac{t^4}{12c^4})) \tag{39}$$

for rectangular airfoils, and for other airfoils given by

$$K_{torsion} = \frac{4I_x}{1 + 16 \frac{I_x}{Ac^2}} \tag{40}$$

where A is the area of the airfoil at the average chord. For hexagonal airfoils,

$$A = 2A_{chamf} + ct \tag{41}$$

For diamond airfoils,

$$A = 2A_{chamf} \tag{42}$$

For biconvex airfoils,

$$A = 4t(\frac{c}{2} - \frac{c}{3}) \tag{43}$$

3.2 Subsonic Flutter Prediction Using the $1/k$ Method

The $1/k$ method was first described by Theodorsen and Garrick in NACA TR496 ([7]), then again in NACA TR685 ([1]), then cited by Fung in his book "An Introduction to the Theory of Aeroelasticity" ([2]). The revolves around k, the reduced frequency, which is defined by

$$k = \frac{\omega b}{V} \tag{44}$$

where ω is the frequency of oscillation, b is the semichord of the fin, and V is the airspeed. *Theodorsen's Circulation Function* (or simply *Theodorsen's Function*) $C(k)$ is defined in *FlutterShy* by

$$C(k) = \frac{H_1^{(2)}(k)}{H_1^{(2)}(k) + iH_0^{(2)}(k)} \tag{45}$$

where $H(k)$ are the Hankel functions of appropriate kind. Further, Theodorsen's function is broken into two parts by

$$C(k) = F(k) + iG(k) \tag{46}$$

equivalently

$$F(k) = \Re(C(k)) \tag{47}$$

$$G(k) = \Im(C(k)) \tag{48}$$

In *FlutterShy*, an array of $1/k$ and k values is generated based upon the input range and resolution for $1/k$, then $C(k)$, $F(k)$, and $G(k)$ arrays are calculated based upon these arrays.

To find the flutter velocity, two quadratic equations are solved, one real and one imaginary, to find the point at which the roots of the two intercept when plotted against $1/k$. In *FlutterShy*, this is done by solving the quadratic equation for both equations at each point of interest. To do so, six components are calculated:

$$A_R = -(\mu + 1) - \frac{2G}{k} \tag{49}$$

$$A_I = \frac{2F}{k} \tag{50}$$

$$B_R = -(\mu\overline{x} - a_h) + \frac{2F}{k^2} - (0.5 - a_h)\frac{2G}{k} \tag{51}$$

$$B_I = \frac{1}{k}(1 + \frac{2G}{k} + (\frac{1}{2} - a_h)2F) \tag{52}$$

$$D_R = -(\mu\overline{x} - a_h) + (0.5 + a_h)\frac{2G}{k} \tag{53}$$

$$D_I = -(0.5 + a_h)\frac{2F}{k} \tag{54}$$

$$E_R = -(\mu\overline{r} + a_h^2 + 0.125) + (0.25 - a_h^2)\frac{2G}{k} - (0.5 + a_h)\frac{2F}{k^2} \tag{55}$$

$$E_I = \frac{1}{k}((0.5 - a_h) - (0.5 + a_h)\frac{2G}{k} - (0.25 - a_h^2)2F) \tag{56}$$

Then the real and imaginary equations are broken into three components each.
For the real equation:

$$\Delta_{R_A} = (1 - g_h g_\alpha)\mu^2\overline{r}^2\frac{\omega_h^2}{\omega_\alpha^2} \tag{57}$$

$$\Delta_{R_B} = \mu\frac{\omega_h^2}{\omega_\alpha^2}(E_R - g_h E_R) + \mu\overline{r}^2(A_R - g_\alpha A_I) \tag{58}$$

$$\Delta_{R_C} = A_R E_R - B_R D_R - A_I E_I + B_I D_I \tag{59}$$

Then for the imaginary equation:

$$\Delta_{I_A} = (g_h + g_\alpha)\mu^2\bar{r}^2\frac{\omega_h^2}{\omega_\alpha^2} \tag{60}$$

$$\Delta_{I_B} = \mu\frac{\omega_h^2}{\omega_\alpha^2}(g_h E_R + E_I) + \mu\bar{r}^2(A_I + g_\alpha A_R) \tag{61}$$

$$\Delta_{I_C} = A_I E_R - B_R D_I + A_R E_I - B_I D_R \tag{62}$$

The roots for the real equation are calculated by

$$X_{R_{1,2}} = \frac{\Delta_{R_B} \pm \sqrt{\Delta_{R_B}^2 - 4\Delta_{R_A}\Delta_{R_C}}}{2\Delta_{R_A}} \tag{63}$$

and the roots for the imaginary equation are calculated by

$$X_{I_{1,2}} = \frac{\Delta_{I_B} \pm \sqrt{\Delta_{I_B}^2 - 4\Delta_{I_A}\Delta_{I_C}}}{2\Delta_{I_A}} \tag{64}$$

unless $\Delta_{I_A} = 0$ (equivalently, $g_h = g_\alpha = 0$) then

$$X_{I_{1,2}} = -\frac{\Delta_{I_C}}{\Delta_{I_B}} \tag{65}$$

Complex roots of the real equation are ignored. Then the square root of every root is taken to produce the set of $\sqrt{X}$. This is useful because the following relation can be used to calculate flutter velocity later

$$\sqrt{X} = \frac{\omega_\alpha}{\omega} \tag{66}$$

equivalently,

$$\omega = \frac{\omega_\alpha}{\sqrt{X}} \tag{67}$$

Complex values of $\sqrt{X_I}$ are ignored. The values of $\sqrt{X}$ for each of the equations form horizontal parabolas when plotted against $1/k$. Due to the nature of the discrete calculation, the intercepts are located by finding the closest values between the real and imaginary roots, using

$$X_{ratio_1} = |1 - |\frac{\sqrt{X_{R_1}}}{\sqrt{X_{I_{1,2}}}}|| \tag{68}$$

and

$$X_{ratio_2} = |1 - |\frac{\sqrt{X_{R_2}}}{\sqrt{X_{I_{1,2}}}}|| \tag{69}$$

Using the appropriate real $\sqrt{X_I}$. The minimum value for $X_{ratio_{1,2}}$ is found, and the two are compared, with the lowest determining the point used for flutter calculation. Using the above definitions of ω and k at this point, *FlutterShy* calculates an initial flutter velocity using

$$V_{f_i} = \frac{\omega_\alpha b}{k\sqrt{X}} \tag{70}$$

3.2.1 Subsonic Compressibility Correction

FlutterShy includes four different compressibility corrections for flutter velocities calculated in the "subsonic" portion. By default, *FlutterShy* does not apply a correction. When a correction *is* applied, an initial Mach number, M_{f_i} is calculated based on the speed of sound, a, at the point flutter is calculated, using

$$M_{f_i} = \frac{V_{f_i}}{a} \tag{71}$$

If the "tr685" correction is selected, a corrected Mach number is calculated with

$$M_{f_c} = \sqrt{M_{f_i}^2 \sqrt{1 - \frac{M_{f_i}^4}{4}} - \frac{M_{f_i}^2}{2}} \tag{72}$$

which comes from [1], and will result in imaginary values if $M_{f_i} > 1$.

If the "mat1" correction is selected, a corrected Mach number is calculated with

$$M_{f_c} = \frac{1}{\sqrt{2}} \sqrt{M_{f_i}^2 \sqrt{4 + M_{f_i}^4} - M_{f_i}^2} \tag{73}$$

which does not produce imaginary numbers, but will result in $M_{f_c} > M_{f_i}$ when $M_{f_i} >\sim 1.5$.

If the "mat2" correction is selected, a corrected Mach number is calculated with

$$M_{f_c} = \frac{1}{\sqrt{2}} \sqrt{M_{f_i}^2 \sqrt{4 + M_{f_i}^4} - M_{f_i}^2} \tag{74}$$

where $M_{f_i} >= 1$ and

$$M_{f_c} = \frac{1}{\sqrt{2}} \sqrt{M_{f_i}^2 \sqrt{4 + M_{f_i}^4} - M_{f_i}^2} \tag{75}$$

where $M_{f_i} < 1$.

If the "kearns" correction is selected, the Kearns Supersonic Method (described in section 3.3) will be applied to M_{f_i} with the user-input parameters.

After calculating M_{f_c}, the flutter velocity is obtained using

$$V_{f_c} = M_{f_c} \times a \tag{76}$$

3.3 Supersonic Flutter Prediction Using the Kearns Supersonic Method

The Kearns supersonic method was presented in the paper "Flutter Simulation" ([3]) by J. P. Kearns, as a simplification of the supersonic method developed by Garrick and Rubinow in NACA TR846 ([4]). This method simplifies some of the supersonic aerodynamics using piston theory, producing a significantly simpler calculation method than outlined in the NACA report. However, this comes with some accuracy loss as well as the lack of any structural damping terms, creating a somewhat conservative estimate for flutter velocity. Because of the use of piston theory, the method does not require further compressibility correction to be applied. In *FlutterShy*, the flight mach number, M from the RASAero II simulation is pulled, then the equation is broken into three components for ease of calculation

$$A = \frac{\mu \overline{r}^2 \sqrt{M^2 - 1}}{\overline{x} b} \tag{77}$$

$$N = (1 - \frac{\omega_h^2}{\omega_\alpha^2})^2 + 4 \frac{\overline{x}^2}{\overline{r}^2} \frac{\omega_h^2}{\omega_\alpha^2} \tag{78}$$

$$D = 2(1 + \frac{\omega_h^2}{\omega_\alpha^2}) \tag{79}$$

which are then combined to calculate flutter velocity using

$$V_f = \omega_\alpha b \sqrt{A \frac{N}{D}} \tag{80}$$

References

[1] Theodorsen, T., and Garrick, I. E., "Mechanism of Flutter A Theoretical and Experimental Investigation of the Flutter Problem," Tech. Rep. TR-685, NACA, 1940. URL https://ntrs.nasa.gov/citations/19930091762.

[2] Fung, Y. C., *An Introduction to the Theory of Aeroelasticity*, Dover, 1993.

[3] Kearns, J. P., "Flutter Simulation," Tech. rep., The John Hopkins University Applied Physics Laboratory, 1962. URL https://apps.dtic.mil/sti/citations/AD0650981.

[4] Garrick, I. E., and Rubinow, S. I., "Flutter and oscillating air-force calculations for an airfoil in two-dimensional supersonic flow," Tech. Rep. TR-846, NACA, 1946. URL https://ntrs.nasa.gov/citations/19930090942.

[5] Blevins, R. D., *Formulas for Natural Frequency and Mode Shape*, Van Nostrand Reinhold Company, 1979, Chap. 8.

[6] Warren C. Young, R. G. B., *Roark's Formulas for Stress and Strain*, McGraw-Hill, 2002.

[7] Theodorsen, T., "General Theory of Aerodynamic Instability and the Mechanism of Flutter," Tech. Rep. TR-496, NACA, 1949. URL https://ntrs.nasa.gov/citations/19930090935.